Case Study Modelling

Table of contents

Data Cleaning . . . 2
EDA . . . 4
Predictive Modelling . . . 10
Reuse the team's cleaned data . . . 10
Common validation and model interfaces . . . 12
Family registry: add future modelling here . . . 13
Common evaluation and selected specifications . . . 17
Prediction diagnostics . . . 20
Scientific Inference . . . 25
Shared bootstrap inference . . . 26
Fitted relationships and response-specific interpretation . . . 28
Prediction exports and team decision . . . 37
Extension and review checklist . . . 38

This working notebook retains the team's data cleaning, EDA and initial linear fits. The Predictive Modelling and Scientific Inference sections add Abbas's spline and random-forest comparison, with a common linear reference. Additional families can be registered at the marked extension point; they reuse the same folds, evaluation, diagnostics, inference and exports. This is the modelling notebook, not the final research report.

```
required <- c("dplyr", "ranger", "knitr")
missing <- required[!vapply(required, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing)) stop("Install required packages: ", paste(missing, collapse = ", "))

responses <- c("mean", "sd", "skew", "kurt")
response_labels <- c(mean = "Mean", sd = "Standard deviation",
                     skew = "Skewness", kurt = "Kurtosis")
settings <- list(cv_seed = 325L, folds = 5L, trees = 500L,
                 bootstrap_seed = 2026L,
                 bootstrap_replicates = as.integer(params$bootstrap_replicates))
```

```r
if (isTRUE(params$run_inference) && settings$bootstrap_replicates < 100L)
  stop("Use at least 100 replicates for the exploratory percentile intervals.")
results_dir <- params$results_dir
dir.create(results_dir, recursive = TRUE, showWarnings = FALSE)
write_result <- function(x, filename) {
  write.csv(x, file.path(results_dir, filename), row.names = FALSE)
}
show_table <- function(x, digits = 4, caption = NULL) {
  knitr::kable(x, digits = digits, row.names = FALSE, caption = caption)
}
# Existing cleaning and EDA below use dplyr.
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

Data Cleaning

```r
data_train <- read.csv(file.path(params$data_dir, "data-train.csv"))
data_test <- read.csv(file.path(params$data_dir, "data-test.csv"))
```

```r
summary(data_train)
```

```
       St               Re               Fr          R_moment_1
 Min.   :0.0500   Min.   : 90.0   Min.   :0.052   Min.   :0.000222
 1st Qu.:0.3000   1st Qu.: 90.0   1st Qu.:0.052   1st Qu.:0.002157
 Median :0.7000   Median :224.0   Median :0.300   Median :0.002958
 Mean   :0.8596   Mean   :214.5   Mean   :  Inf   Mean   :0.040394
 3rd Qu.:1.0000   3rd Qu.:224.0   3rd Qu.:  Inf   3rd Qu.:0.087868
 Max.   :3.0000   Max.   :398.0   Max.   :  Inf   Max.   :0.172340
```

```
  R_moment_2          R_moment_3        R_moment_4
 Min.   :   0.0001   Min.   :      0   Min.   :0.000e+00
 1st Qu.:   0.0245   1st Qu.:      0   1st Qu.:3.000e+00
 Median :   0.0808   Median :      1   Median :2.100e+01
 Mean   :  92.4902   Mean   : 753370   Mean   :6.194e+09
 3rd Qu.:   0.5345   3rd Qu.:     40   3rd Qu.:5.345e+03
 Max.   :1044.3000   Max.   :9140000   Max.   :8.000e+10
```

X = Predictors: St, Re, Fr

y = Output : probability distribution, or

Y = first four raw moments:

- R_moment_1 E[X]
- R_moment_2 E[X2]
- R_moment_3 E[X3]
- R_moment_4 E[X4]

But, we want central moments, so actual response variables are:

- Mean: = E(X). This measures a distribution's centrality.
- Standard deviation: $\sigma = \sqrt{E[(X-\mu)^2]}$. This measures variability around the mean.
- Skewness: = E[(X −)3]/ 3. This measures a distribution's degree of asymmetry.
- Kurtosis: = E[(X −)4]/ 4. This measures how "heavy" a distribution's tails are.

```
#data cleaning + transforming
cleaned_data_train <- data_train |> mutate(
  mean = R_moment_1,
  var = R_moment_2 - R_moment_1^2, #variance = E[X^2] - (E[X])^2
  sd = sqrt(var), #sd = root(var)
  c3 = R_moment_3 - mean^3 - 3*mean*R_moment_2 + 3*(mean^2)*R_moment_1, # expanded using line
  c4 = R_moment_4 - 4*R_moment_1*R_moment_3 + 6*R_moment_1^2*R_moment_2 - 3*R_moment_1^4, # e
  skew = c3 / sd^3,
  kurt = c4 / sd^4
) |>
  select(St, Re, Fr, mean, sd, skew, kurt)
```

```r
#transform training + test data
transform_preds <- function(df) {
  df |> mutate(
    Fr_cat = factor(Fr, levels = c(Inf, 0.3, 0.052),
                    labels = c("none", "moderate", "strong")),
    Re_cat = factor(Re, levels = c(90, 224, 398))
    )
}

cleaned_data_train <- transform_preds(cleaned_data_train)
cleaned_data_test <- transform_preds(data_test)
```

```r
table(cleaned_data_train$Re_cat, cleaned_data_train$Fr_cat)
```

```
     none moderate strong
  90    9       10     12
  224  12       13     14
  398   9        0     10
```

```r
head(cleaned_data_train)
```

```
    St  Re    Fr       mean         sd      skew       kurt   Fr_cat Re_cat
1 0.10 224 0.052 0.00215700 0.36103372 305.42800 93371.6556   strong    224
2 3.00 224 0.052 0.00379030 0.68586123 216.76225 47012.2515   strong    224
3 0.70 224   Inf 0.00290540 0.20854390  90.58974  8216.7778     none    224
4 0.05  90   Inf 0.06352800 0.29430799  17.67980   420.2523     none     90
5 0.70 398   Inf 0.00036945 0.07889273 257.58554 66372.7845     none    398
6 2.00  90 0.300 0.14780000 1.40888437  12.64607   165.0999 moderate     90
```

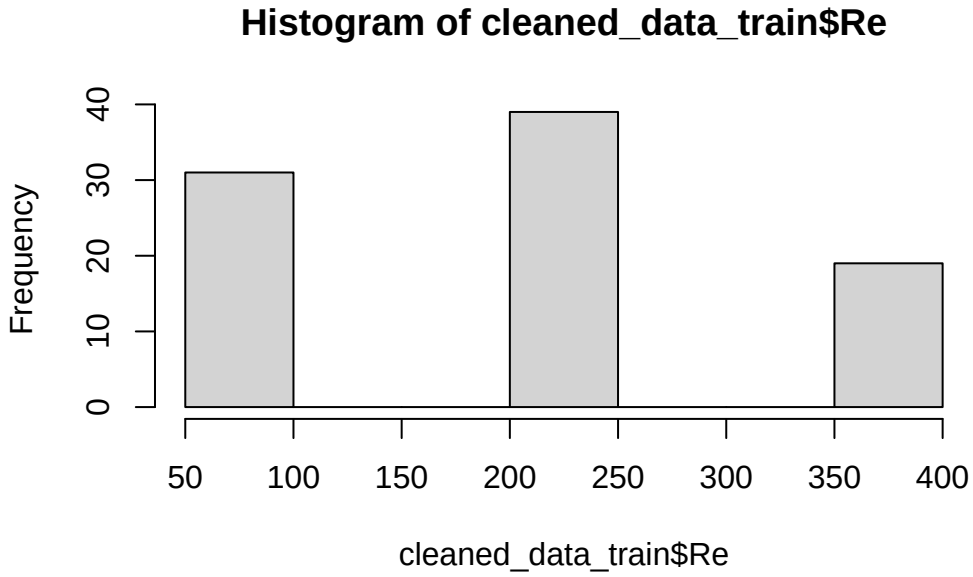
EDA

- Individual ranges

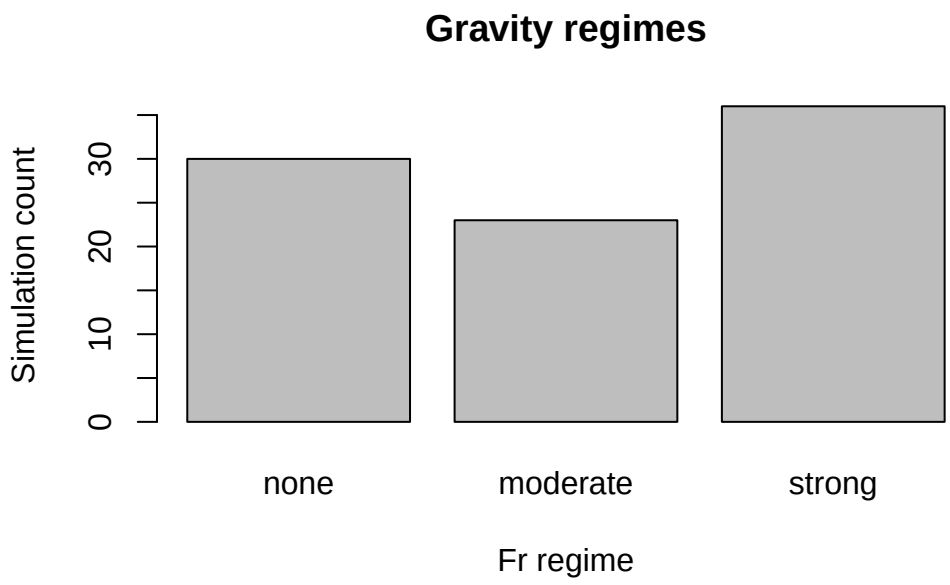
```r
hist(cleaned_data_train$St)
```




```
hist(cleaned_data_train$Re)
```



```
barplot(table(cleaned_data_train$Fr_cat),
        main = "Gravity regimes", xlab = "Fr regime", ylab = "Simulation count")
```




```
unique(cleaned_data_train$Fr)
```

```
[1] 0.052   Inf 0.300
```

The data include three Re levels and three meaningful Fr regimes. In particular, `Fr = Inf` is the no-gravity limit, not an impossible or missing measurement. Smaller Fr represents stronger gravitational influence in this study. We retain all regimes and use the factors prepared above. The categorical representation matches the finite simulation levels and test design; large numerical values alone would not justify treating a variable as a factor.

St describes particle inertia and is kept numeric. The slides consider values up to 3; modelling below examines whether the response relationships are linear or curved.

```
#Re & Fr values remain same in train + test, so consider them factors
unique(cleaned_data_train$Re); unique(cleaned_data_train$Fr);
```

```
[1] 224  90 398
```

```
[1] 0.052   Inf 0.300
```

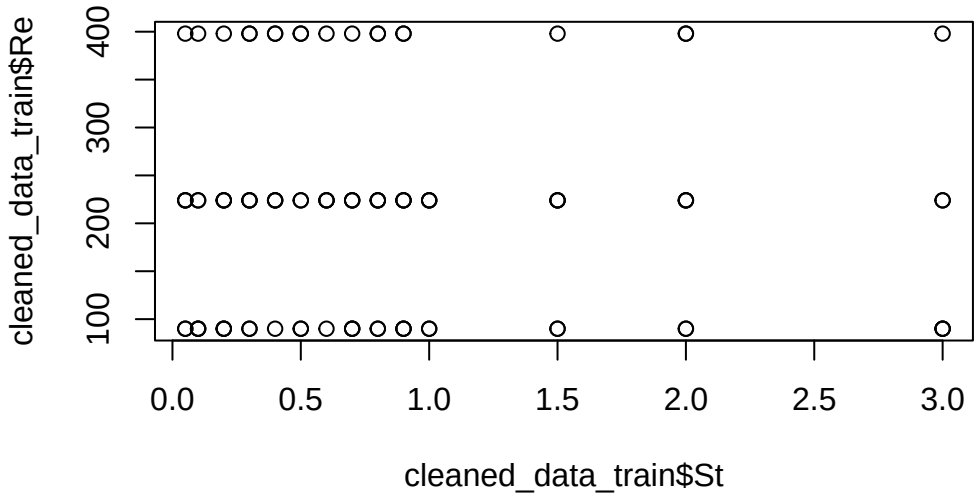
```
unique(data_test$Re); unique(data_test$Fr)
```

```
[1] 398 224  90
```

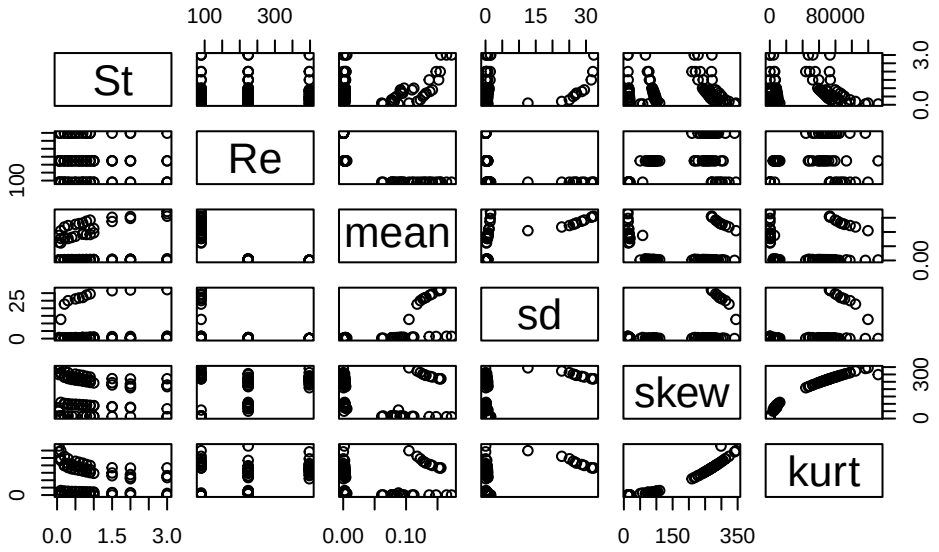
```
[1] 0.052   Inf 0.300
```

1. Interactions

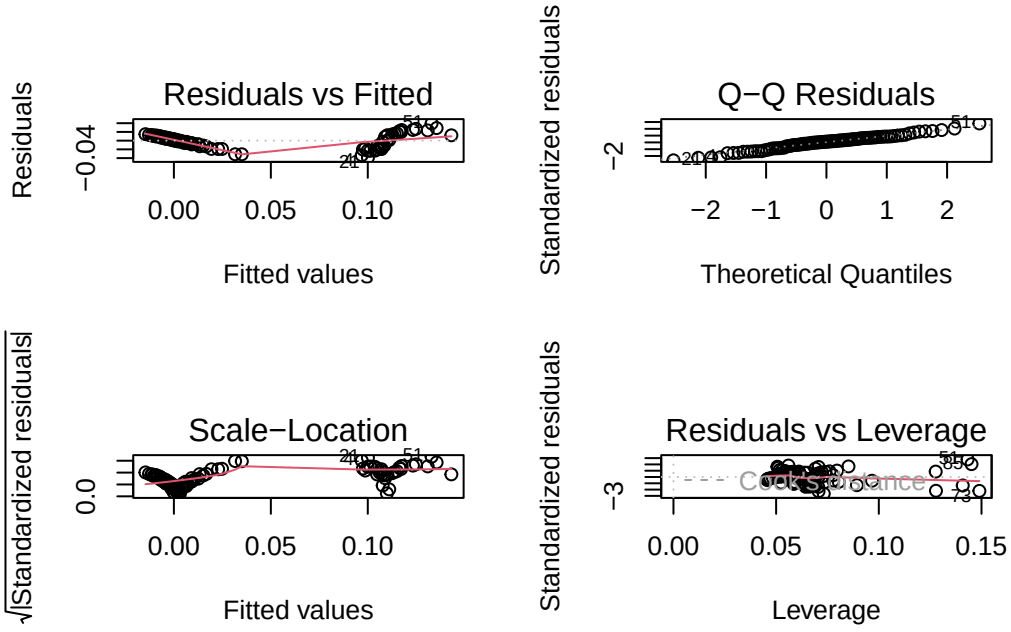
```
plot(cleaned_data_train$St, cleaned_data_train$Re)
```




```
pairs(cleaned_data_train[c("St", "Re", "mean", "sd", "skew", "kurt")])
```




```
fit <- lm(mean ~ Re_cat + Fr_cat + St, data = cleaned_data_train)
par(mfrow = c(2, 2)); plot(fit)
```



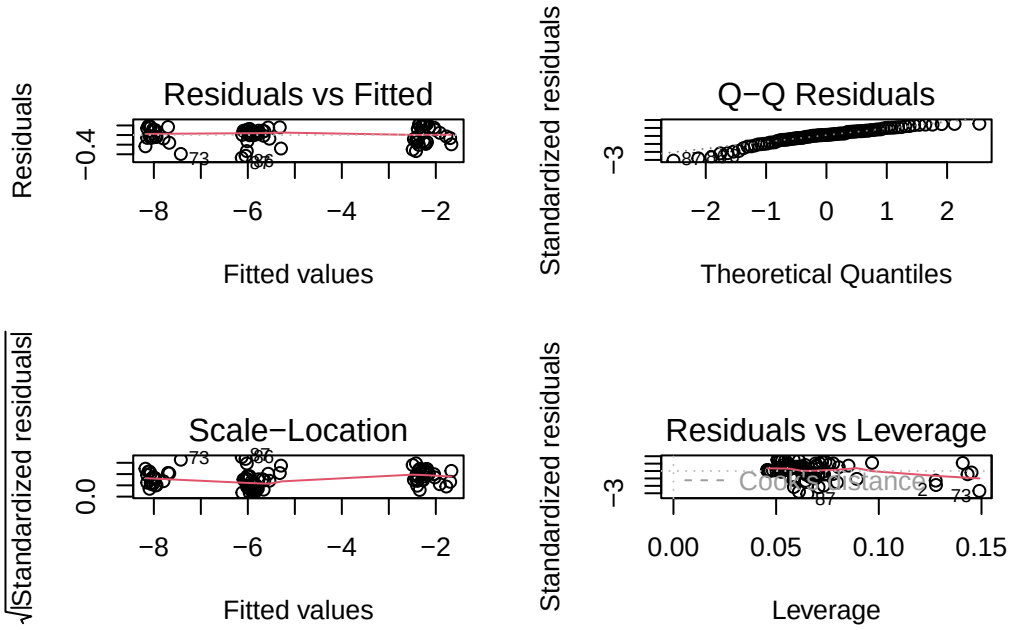
```
par(mfrow = c(1, 1))
```

The initial linear diagnostics motivate checking a response transformation and more flexible specifications. The shared CV comparison below evaluates those choices rather than assuming a transformation will improve prediction:

- fitted vals for mean are negative, so predicting log(mean) might be easier

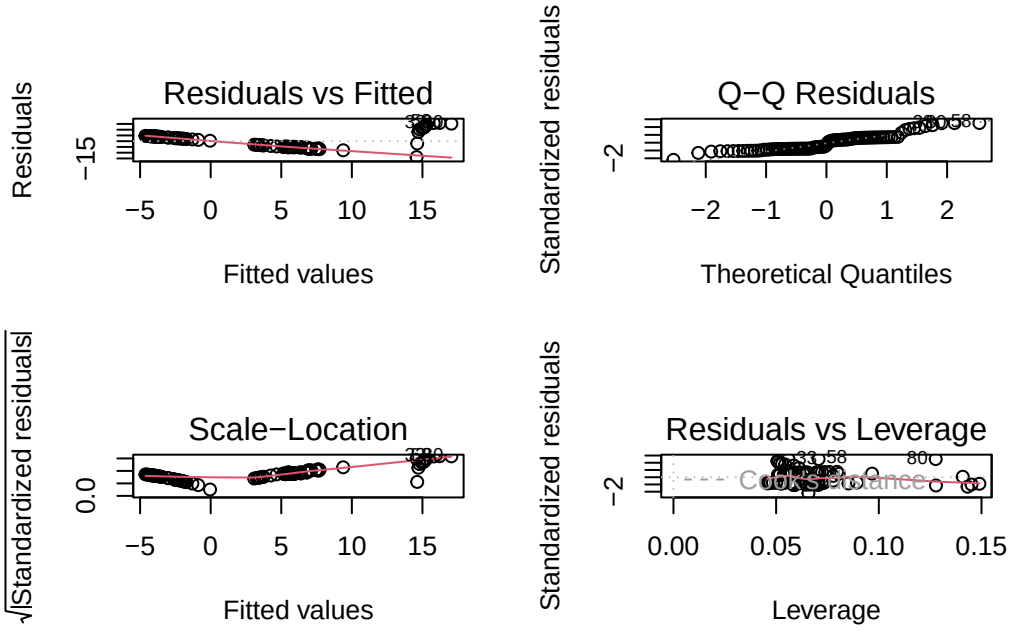
```
cleaned_data_train <- cleaned_data_train |>mutate(log_mean = log(mean))
```

```
fit2 <- lm(log_mean ~ Re_cat + Fr_cat + St, data = cleaned_data_train)
par(mfrow = c(2, 2)); plot(fit2)
```




```
par(mfrow = c(1, 1))
```

```
fit_sd <- lm(sd ~ Re_cat + Fr_cat + St, data = cleaned_data_train)
par(mfrow = c(2, 2)); plot(fit_sd)
```




```
par(mfrow = c(1, 1))
```

Predictive Modelling

Questions guiding the comparison are whether transformations improve prediction, whether nonlinear St effects help relative to a simple linear reference, and whether gravity-specific St relationships are supported. We fit each of the four summaries separately within a family and select settings using common CV folds. The earlier exploratory fits above remain available for comparison; the reference below uses joint Re–Fr group intercepts, so it is a distinct, explicitly stated specification.

Reuse the team's cleaned data

The original cleaning formulas above produce the required mean, SD, skewness and ordinary kurtosis. No duplicate response transformation pipeline is introduced. The aliases below provide one stable interface for every registered model family. Raw moment columns are never used as predictors.

```
train_raw <- data_train
test_raw <- data_test
train <- cleaned_data_train
```

```
test <- cleaned_data_test
stopifnot(!anyNA(train_raw), !anyNA(test_raw),
          !anyDuplicated(train_raw[c("St", "Re", "Fr")]),
          all(is.finite(as.matrix(train[responses]))),
          all(as.matrix(train[responses]) > 0))
train$group <- interaction(train$Re_cat, train$Fr_cat, drop = TRUE)
test$group <- interaction(test$Re_cat, test$Fr_cat, drop = TRUE)
stopifnot(all(as.character(test$group) %in% levels(train$group)))
test$group <- factor(test$group, levels = levels(train$group))
input_files <- file.path(params$data_dir, c("data-train.csv", "data-test.csv"))
write_result(data.frame(file = basename(input_files), md5 = unname(tools::md5sum(input_files)
             "input-checksums.csv")
```

There are 89 labelled simulations and 23 test settings. The $\mathrm{Re} = 398$, $\mathrm{Fr} = 0.3$ regime is absent. Inference below is restricted to supported regimes. The response ranges and boundary checks are recorded before fitting.

```
ranges <- do.call(rbind, lapply(responses, function(y) {
  data.frame(response = response_labels[y], minimum = min(train[[y]]),
             median = median(train[[y]]), maximum = max(train[[y]]))
}))
show_table(ranges, digits = 6)
```

response	minimum	median	maximum
Mean	0.000222	0.002958	0.17234
Standard deviation	0.010060	0.284225	32.31526
Skewness	11.973037	110.116336	344.90546
Kurtosis	150.533639	12158.719840	132136.73122

```
support <- do.call(rbind, lapply(seq_len(nrow(test)), function(i) {
  observed <- train$St[train$group == test$group[i]]
  data.frame(test_row = i, test_raw[i, c("St", "Re", "Fr")],
             outside_group_St_range = test$St[i] < min(observed) |
                                      test$St[i] > max(observed))
}))
write_result(support, "test-support.csv")
show_table(support[support$outside_group_St_range, ],
           caption = "Test settings outside their group's observed St range.")
```

Table 2: Test settings outside their group's observed St range.

test_row	St	Re	Fr	outside_group_St_range
1	0.05	398	0.052	TRUE
9	3.00	398	Inf	TRUE
10	3.00	224	0.300	TRUE
15	0.05	90	0.300	TRUE

Positive responses span substantially different scales. We consider raw and log response models rather than assuming one transformation is suitable for every outcome. A log fit is converted back by exponentiation. This predicts a fitted geometric-scale response, not an automatically bias-corrected conditional arithmetic mean. All errors below are calculated after returning to the original response scale.

Common validation and model interfaces

We use ordinary five-fold cross-validation, with identical folds for every family and response. Each candidate is fitted using four folds and predicts the fifth. Spline knots and parameters are learned from that training subset only. Candidate selection minimises original-scale MSE separately for each response. RMSE is its square root. Candidates that fail fitting or produce nonpositive validation predictions are recorded and excluded because these responses are positive in the supplied dataset.

These are **CV model-selection estimates**, not independent final test errors. Selecting the lowest score can introduce optimism. Folds overlap in their training data, and fold wins are descriptive rather than independent significance tests. No hidden test response is used for tuning. Comparisons concern the observed regimes; they do not establish performance at new Re or Fr levels.

```
set.seed(settings$cv_seed)
fold <- sample(rep(seq_len(settings$folds), length.out = nrow(train)))
stopifnot(all(vapply(seq_len(settings$folds), function(k)
  all(table(train$group[fold != k]) > 0), logical(1))))
write_result(data.frame(row = seq_len(nrow(train)), fold = fold), "cv-folds.csv")

prepare_target <- function(d, response, scale) {
  d$target <- if (scale == "log") log(d[[response]]) else d[[response]]
  d
}
fit_regression <- function(d, response, cfg, formula) {
  d <- prepare_target(d, response, cfg$scale)
```

```
  # Bind the spline df to the formula environment for safe prediction later.
  model <- lm(formula, data = d)
  if (model$rank < length(coef(model))) stop("Rank-deficient regression fit")
  list(model = model, scale = cfg$scale)
}
predict_regression <- function(object, newdata) {
  z <- as.numeric(predict(object$model, newdata = newdata))
  if (object$scale == "log") exp(z) else z
}
fit_forest <- function(d, response, cfg, seed) {
  d <- prepare_target(d, response, cfg$scale)
  model <- ranger::ranger(target ~ St + Re_cat + Fr_cat, data = d,
    num.trees = settings$trees, mtry = cfg$mtry, min.node.size = cfg$node,
    respect.unordered.factors = "partition", importance = "impurity",
    num.threads = 1L, seed = seed)
  list(model = model, scale = cfg$scale)
}
predict_forest <- function(object, newdata) {
  z <- as.numeric(predict(object$model, data = newdata)$predictions)
  if (object$scale == "log") exp(z) else z
}
```

Family registry: add future modelling here

Each registered family provides a candidate grid, a fit function, and a prediction function returning **one original-scale value per row**. The shared engine then performs CV, selects settings, refits, diagnoses and exports predictions. Main families also enter the common bootstrap and inference sections automatically.

Linear reference

The fixed reference is $\log(Y) = \alpha_{g(Re,Fr)} + \beta St + \epsilon$. It supplies a simple comparison, rather than presuming nonlinear models improve every response. Group intercepts permit joint differences between the observed Re–Fr regimes.

```
families <- list()
families$linear <- list(
  label = "Linear reference", role = "baseline",
  description = "Fixed log-linear regression with group intercepts and a straight St effect.
  grid = data.frame(scale = "log"),
  fit = function(d, response, cfg, seed)
```

```
    fit_regression(d, response, cfg, target ~ group + St),
  predict = predict_regression
)
```

Natural cubic spline regression

For each response we consider a shared St shape,

$$h(Y) = \alpha_{g(Re,Fr)} + f(St) + \epsilon,$$

or gravity-specific St shapes,

$$h(Y) = \alpha_{g(Re,Fr)} + f_{Fr}(St) + \epsilon,$$

where h is identity or log. We compare 2, 3 and 4 spline degrees of freedom. The gravity-specific option shares its shape across Re within a gravity level. It is not eight unrelated curves. Natural splines are linear beyond their boundary knots on the modelled scale; that restriction does not guarantee extrapolation accuracy. Interpret fitted functions and contrasts rather than individual basis coefficients. A selected interaction is not, by itself, a significance test.

```
families$spline <- list(
  label = "Natural cubic spline", role = "main",
  description = "Group intercepts, controlled smooth St effects, and optional gravity-specif
  grid = expand.grid(df = c(2L, 3L, 4L), shape = c("shared", "gravity"),
                     scale = c("log", "raw"), stringsAsFactors = FALSE),
  fit = function(d, response, cfg, seed) {
    df_value <- cfg$df
    formula <- if (cfg$shape == "shared")
      target ~ group + splines::ns(St, df = df_value) else
      target ~ group + splines::ns(St, df = df_value) +
               splines::ns(St, df = df_value):Fr_cat
    fit_regression(d, response, cfg, formula)
  },
  predict = predict_regression
)
```

Random forest regression

The forest averages $B = 500$ regression trees, $\hat{f}(x) = B^{-1}\sum_{b=1}^{B}\widehat{T}_b(x)$. Each tree uses a bootstrap sample; each split considers a random subset of the three predictors. We compare one or two candidate predictors per split, minimum node-size settings 3 or 8, and raw or log responses. Considering all three predictors would be bagging under the lecture's distinction, so that option is excluded here. Tree splits can capture interactions without a prescribed interaction formula. Interpretation uses fitted comparisons and descriptive RSS importance, not causal effects.

```
families$forest <- list(
  label = "Random forest", role = "main",
  description = "An average of regression trees with randomised split predictors; flexible in
  grid = expand.grid(mtry = c(1L, 2L), node = c(3L, 8L),
                     scale = c("log", "raw"), stringsAsFactors = FALSE),
  fit = fit_forest, predict = predict_forest
)
```

Quadratic Regression

To investigate whether a simple nonlinear St effect can improve prediction relative to the linear reference, we fit quadratic regression models. For each response, we consider the specification

$$h(Y) = \alpha_{g(Re,Fr)} + \beta_1 St + \beta_2 St^2 + \epsilon$$

where (h) is either identity or log. Group intercepts account for differences across observed Re–Fr regimes, while the quadratic term allows the response to vary nonlinearly with St. This provides a simpler alternative to natural cubic splines, which allow more flexible nonlinear relationships. The St relationship is shared across regimes, so this model does not introduce additional St interactions. Both response scales are evaluated using the same five-fold cross-validation procedure, with model selection based on original-scale MSE.

```
families$quadratic <- list(
  label = "Quadratic regression", role = "main",
  description = "Group intercepts with a quadratic St effect.",
  grid = data.frame(scale = c("raw", "log")),
  fit = function(d, response, cfg, seed)
    fit_regression(d, response, cfg,
                   target ~ group + St + I(St^2)),
  predict = predict_regression
)
```

Generalized additive regression

To investigate whether a more flexible but interpretable nonlinear model can improve predictive performance, we consider generalized additive models (GAMs). For each response, we fit

$$h(Y) = \alpha_{g(Re,Fr)} + f(St) + \epsilon$$

where (h) is either identity or log, and (f(St)) is a penalized smooth function estimated from the training data. Group intercepts account for differences across the observed Re–Fr regimes, while the smooth term captures potentially nonlinear St effects without imposing a fixed polynomial form.

Smoothness is estimated using restricted maximum likelihood (REML), with a limited spline basis to control complexity given the small sample size. As with the other model families, we compare raw and log response scales using the same five-fold cross-validation procedure and original-scale MSE. The shared St smooth provides a direct comparison with the existing spline specifications while allowing data-adaptive regularization of the nonlinear relationship.

```
families$gam <- list(
  label = "Generalized additive model",
  role = "main",
  description = "Group intercepts with a penalized smooth St effect.",

  grid = data.frame(
    scale = c("raw", "log")
  ),

  fit = function(d, response, cfg, seed) {
    d <- prepare_target(d, response, cfg$scale)

    model <- mgcv::gam(
      target ~ group + s(St, k = 5),
      data = d,
      method = "REML"
    )

    list(model = model, scale = cfg$scale)
  },

  predict = function(object, newdata) {
    z <- as.numeric(
      predict(object$model, newdata = newdata)
    )
```

```
    if (object$scale == "log") exp(z) else z
  }
)
```

Common evaluation and selected specifications

```
configuration_text <- function(cfg) paste(names(cfg), as.character(cfg[1, ]),
                                          sep = "=", collapse = "; ")
evaluate_family <- function(id, family) {
  output <- list()
  for (response in responses) {
    grid <- family$grid
    scores <- rep(Inf, nrow(grid)); failures <- rep("", nrow(grid))
    p <- matrix(NA_real_, nrow(train), nrow(grid))
    for (j in seq_len(nrow(grid))) {
      for (k in seq_len(settings$folds)) {
        index <- fold == k
        p[index, j] <- tryCatch({
          object <- family$fit(train[!index, ], response, grid[j, , drop = FALSE],
                               settings$cv_seed + k)
          prediction <- family$predict(object, train[index, ])
          if (length(prediction) != sum(index)) stop("Wrong prediction length")
          prediction
        }, error = function(e) {
          failures[j] <<- paste(failures[j], conditionMessage(e), sep = "; ")
          rep(NA_real_, sum(index))
        })
      }
      if (all(is.finite(p[, j])) && all(p[, j] > 0)) {
        scores[j] <- mean((train[[response]] - p[, j])^2)
      } else if (!nzchar(failures[j])) failures[j] <- "Nonfinite or nonpositive validation pr
    }
    if (!any(is.finite(scores))) stop("No valid candidates for ", id, ": ", response)
    best <- which.min(scores); cfg <- grid[best, , drop = FALSE]
    object <- family$fit(train, response, cfg, settings$cv_seed)
    test_prediction <- family$predict(object, test)
    stopifnot(length(test_prediction) == nrow(test),
              all(is.finite(test_prediction)), all(test_prediction > 0))
    output[[response]] <- list(
      configuration = cfg, object = object,
```

```
      candidates = cbind(grid, CV_MSE = scores, status = ifelse(is.finite(scores), "valid", i
      cv_prediction = p[, best], test_prediction = test_prediction,
      score = data.frame(family = id, label = family$label, response = response,
                         CV_MSE = scores[best], CV_RMSE = sqrt(scores[best]),
                         configuration = configuration_text(cfg)))
    write_result(output[[response]]$candidates, paste0("candidates-", id, "-", response, ".cs
  }
  output
}
fits <- lapply(names(families), function(id) evaluate_family(id, families[[id]]))
names(fits) <- names(families)
score_table <- do.call(rbind, lapply(fits, function(family)
  do.call(rbind, lapply(family, function(x) x$score))))
rownames(score_table) <- NULL
cv_predictions <- do.call(rbind, lapply(names(fits), function(id)
  do.call(rbind, lapply(responses, function(y) data.frame(
    family = id, response = y, row = seq_len(nrow(train)), fold = fold,
    observed = train[[y]], predicted = fits[[id]][[y]]$cv_prediction)))))
write_result(score_table, "cv-selection-scores.csv")
write_result(cv_predictions, "cv-predictions.csv")
show_table(score_table[c("label", "response", "CV_RMSE", "configuration")], digits = 6)
```

label	response	CV_RMSE	configuration
Linear reference	mean	0.011289	scale=log
Linear reference	sd	4.693302	scale=log
Linear reference	skew	36.136061	scale=log
Linear reference	kurt	16243.859932	scale=log
Natural cubic spline	mean	0.003965	df=2; shape=gravity; scale=log
Natural cubic spline	sd	2.403019	df=4; shape=shared; scale=log
Natural cubic spline	skew	36.492996	df=2; shape=shared; scale=log
Natural cubic spline	kurt	16341.691352	df=2; shape=shared; scale=log
Random forest	mean	0.010027	mtry=2; node=3; scale=raw
Random forest	sd	2.565200	mtry=2; node=3; scale=raw
Random forest	skew	31.810132	mtry=2; node=3; scale=raw
Random forest	kurt	14137.323231	mtry=2; node=8; scale=raw
Quadratic regression	mean	0.009161	scale=log
Quadratic regression	sd	2.993984	scale=log
Quadratic regression	skew	36.437121	scale=log
Quadratic regression	kurt	16310.248057	scale=log
Generalized additive model	mean	0.009893	scale=log
Generalized additive model	sd	3.064577	scale=log

label	response	CV_RMSE	configuration
Generalized additive model	skew	36.136066	scale=log
Generalized additive model	kurt	16261.100024	scale=log

Compare errors **within a response** because the four summaries have different units and scales. A larger kurtosis RMSE does not mean kurtosis is intrinsically less accurately predicted than mean. More complex families need not beat the linear reference. Individual fold wins and relative errors can be explored below; neither establishes statistical significance.

Quadratic regression comparison

Quadratic regression improved prediction over the linear reference for mean and standard deviation, reducing CV RMSE from 0.011289 to 0.009161 and from 4.693302 to 2.993984, respectively. However, natural cubic splines continued to outperform quadratic regression for both responses, suggesting that additional flexibility in the St relationship, and potentially gravity-specific effects, are useful for prediction.

For skewness and kurtosis, quadratic regression offered no meaningful improvement over the linear reference and was outperformed by random forests. Diagnostic plots also showed residual patterns and several poorly predicted observations. Thus, quadratic regression provides evidence that simple curvature alone is insufficient to achieve the best predictive performance across the four responses. We retain it as a comparison model but do not select it as the preferred specification for any response.

Generalized additive model comparison

Generalized additive models were evaluated to determine whether penalized smooth St effects could improve prediction while retaining an interpretable regression structure. We considered both raw and log response scales, with the log scale selected for all four responses.

The GAM did not outperform the existing best-performing models. For mean and standard deviation, GAM CV RMSEs were 0.009893 and 3.064577, compared with 0.003965 and 2.403019 for natural cubic splines. For skewness and kurtosis, GAM CV RMSEs were 36.136066 and 16261.100024, compared with 31.810132 and 14137.323231 for random forests.

Residual diagnostics indicated remaining prediction errors, particularly for extreme observations. These findings suggest that the shared smooth St specification does not provide sufficient improvement over the existing approaches. We therefore retain GAMs as comparison models but do not select them as the preferred predictive specification for any response.

```
fold_scores <- do.call(rbind, lapply(split(cv_predictions,
  interaction(cv_predictions$family, cv_predictions$response, cv_predictions$fold)),
  function(d) data.frame(family = d$family[1], response = d$response[1],
    fold = d$fold[1], RMSE = sqrt(mean((d$observed - d$predicted)^2)))))
write_result(fold_scores, "cv-fold-scores.csv")
```

Prediction diagnostics

Held-out predictions should lie near the diagonal. Residuals should be inspected for systematic patterns, unequal spread and unusual cases. Training residuals are descriptive fit diagnostics; the CV predictions supply the out-of-fold comparison. We do not discard unusual simulation settings automatically.

```
for (id in names(families)) {
  par(mfrow = c(2, 4), mar = c(4, 4, 3, 1))
  for (y in responses) {
    p <- fits[[id]][[y]]$cv_prediction; limits <- range(c(train[[y]], p))
    plot(train[[y]], p, log = "xy", xlim = limits, ylim = limits, pch = 16,
         cex = .6, col = "#146c94", xlab = "Observed", ylab = "CV prediction",
         main = paste(families[[id]]$label, y)); abline(0, 1, lty = 2)
  }
  for (y in responses) {
    selected <- fits[[id]][[y]]
    p <- families[[id]]$predict(selected$object, train)
    scale <- if ("scale" %in% names(selected$configuration)) selected$configuration$scale els
    observed <- train[[y]]
    if (scale == "log") { p <- log(p); observed <- log(observed) }
    plot(p, observed - p, pch = 16, cex = .6, col = "#146c94",
         xlab = paste("Fitted (", scale, "scale)"), ylab = "Residual",
         main = paste(families[[id]]$label, y)); abline(h = 0, lty = 2)
  }
}
```

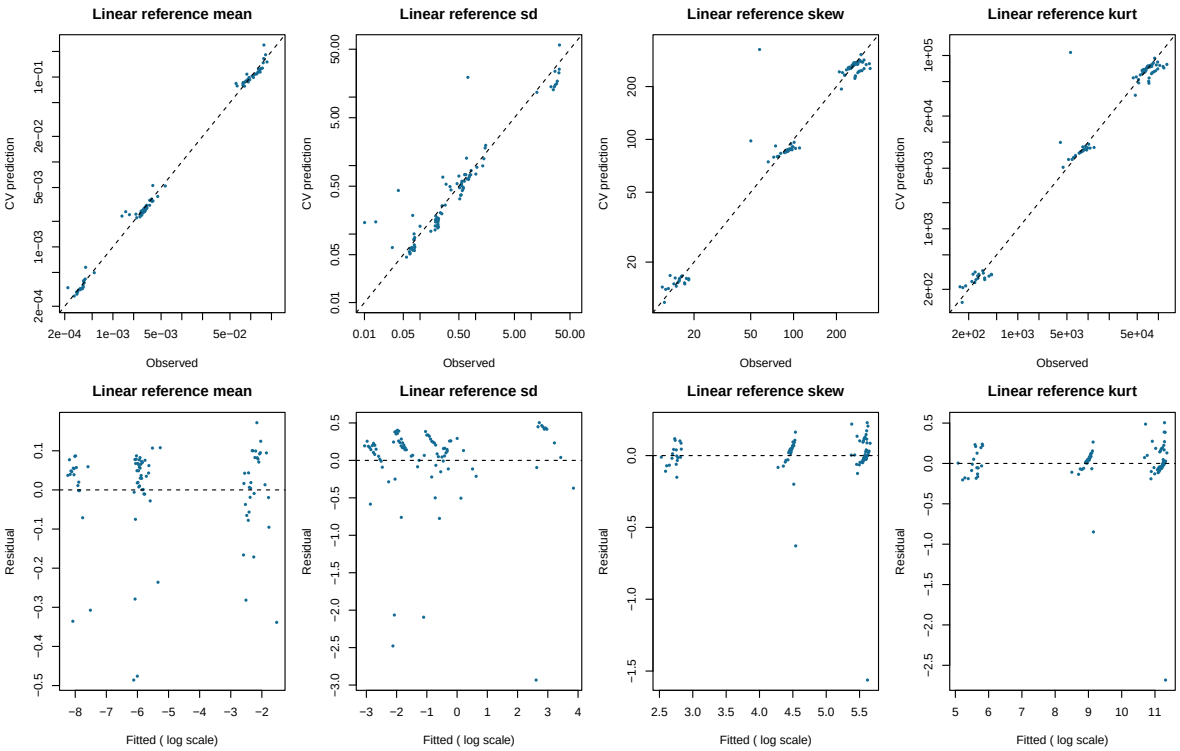


Figure 1: Held-out predictions (top row) and training residuals (bottom row), by response. Each panel identifies its family.

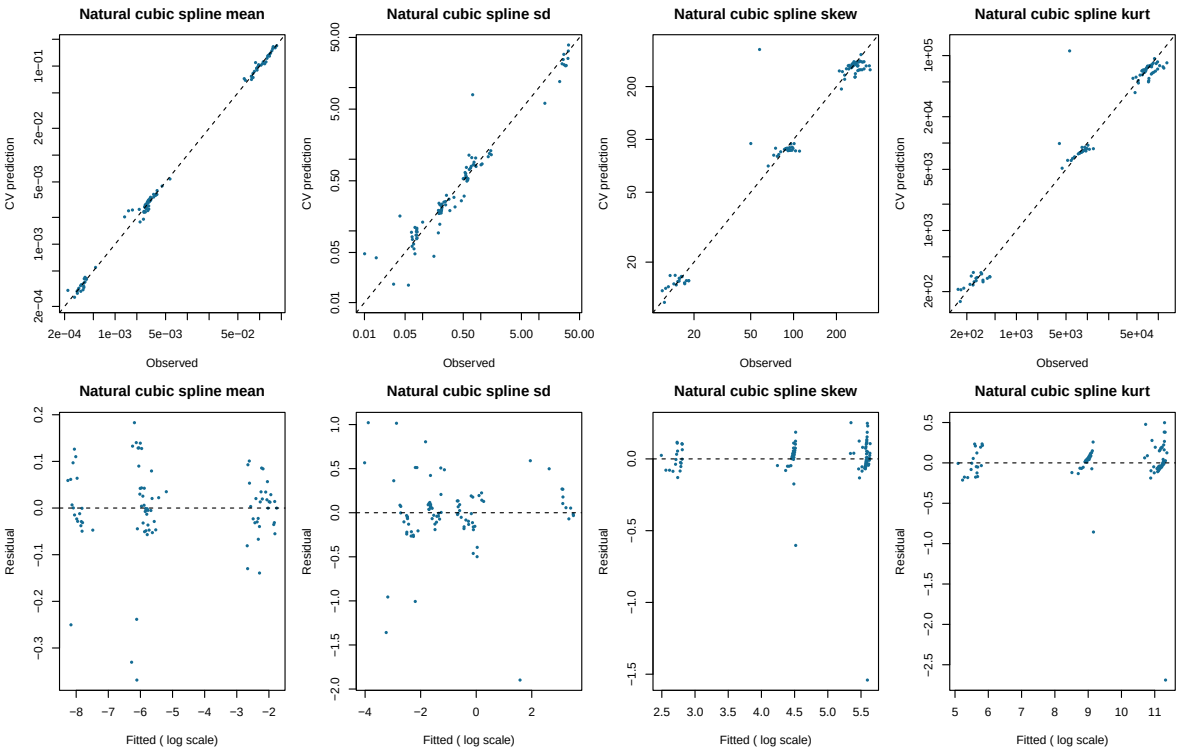


Figure 2: Held-out predictions (top row) and training residuals (bottom row), by response. Each panel identifies its family.

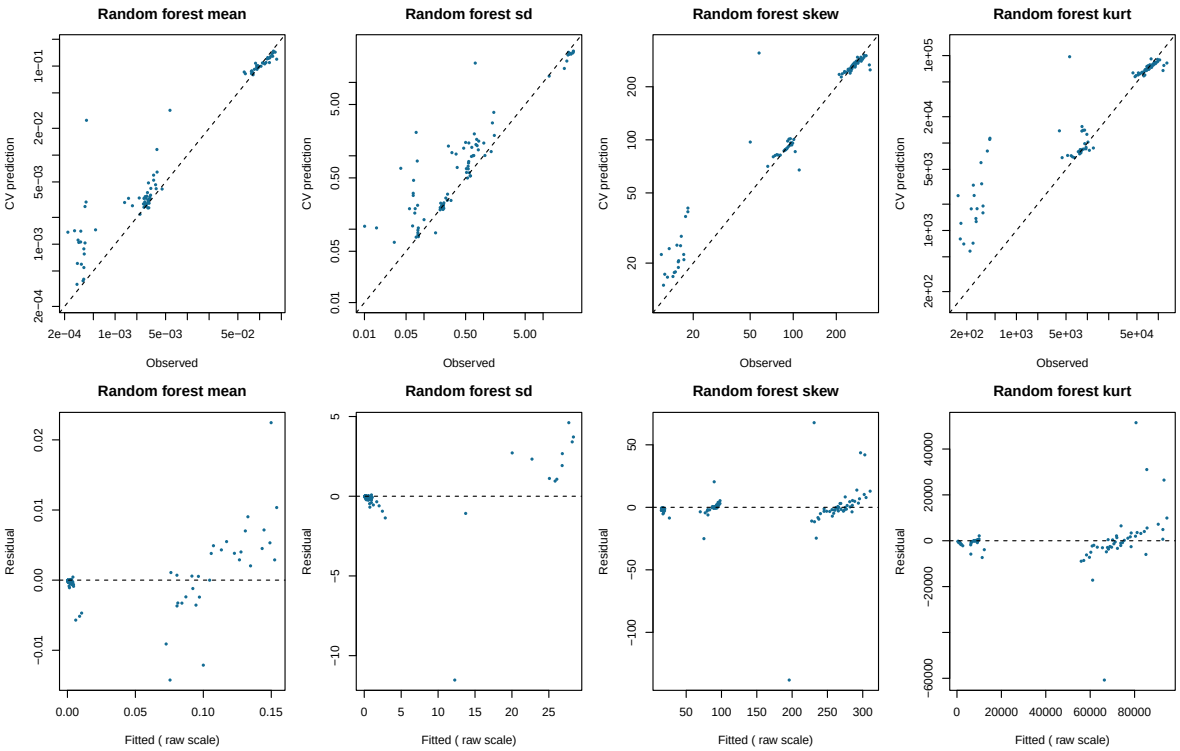


Figure 3: Held-out predictions (top row) and training residuals (bottom row), by response. Each panel identifies its family.

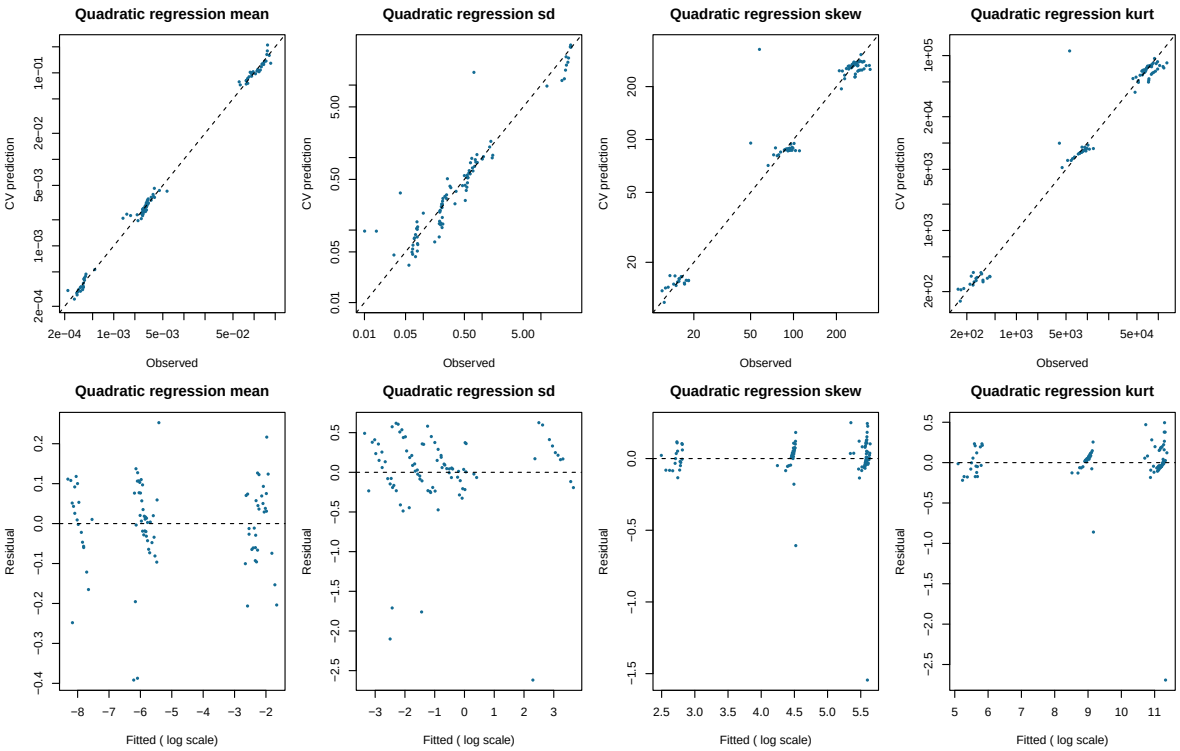


Figure 4: Held-out predictions (top row) and training residuals (bottom row), by response. Each panel identifies its family.

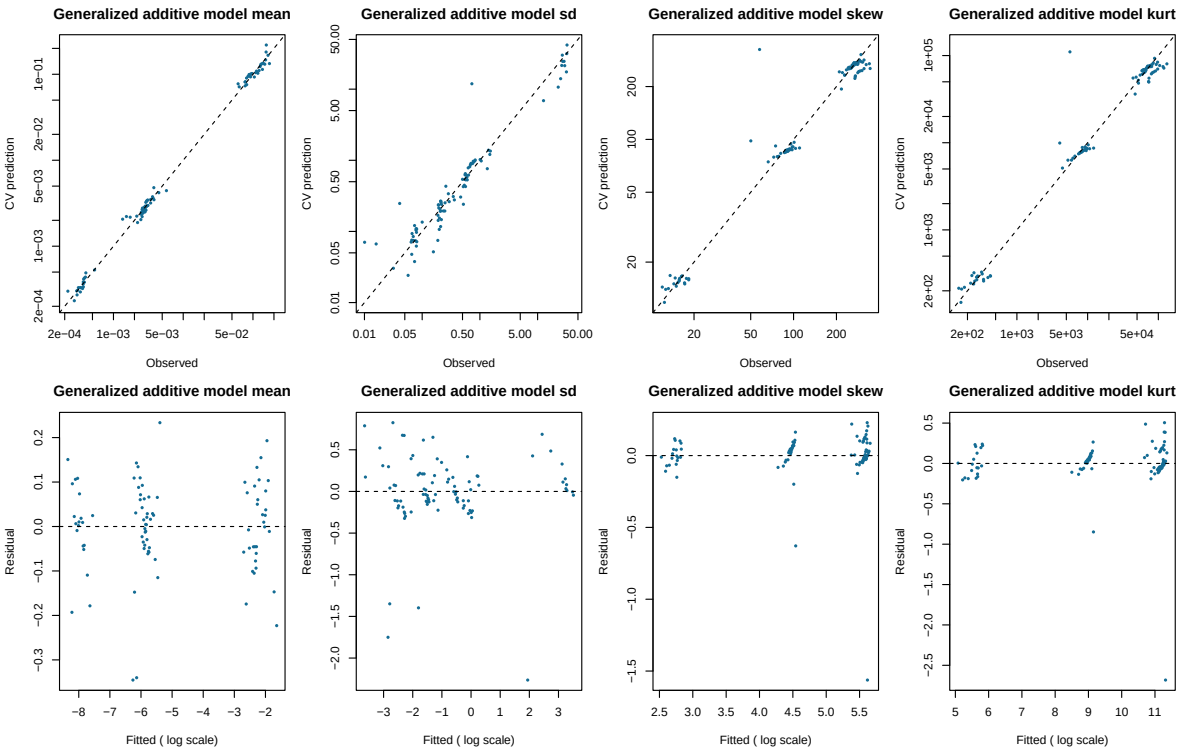


Figure 5: Held-out predictions (top row) and training residuals (bottom row), by response. Each panel identifies its family.

```
par(mfrow = c(1, 1))
```

Across the five evaluated model families, natural cubic spline regression achieved the lowest CV RMSE for mean and standard deviation, whereas random forest regression achieved the lowest CV RMSE for skewness and kurtosis. These results suggest that different distribution summaries benefit from different levels and forms of model flexibility.

The selected spline specifications capture smooth nonlinear St relationships, while random forests allow nonlinearities and interactions without requiring explicit functional forms. We treat these as preliminary selections and examine their diagnostic adequacy, sensitivity to validation splits, and scientific interpretability before finalizing predictions.

Scientific Inference

The scientific questions are how Reynolds number (Re), Froude number (Fr), and Stokes number (St) relate to the particle cluster-volume distribution, whether these relationships are

linear or nonlinear, whether mean, spread, and tail behavior respond similarly, and whether the effects of the input parameters depend on one another.

Following the five-family cross-validation comparison, we focus scientific inference on the best-performing specification for each response. Natural cubic spline regression is selected for mean and standard deviation, with CV RMSEs of 0.003965 and 2.403019, respectively. Random forest regression is selected for skewness and kurtosis, with CV RMSEs of 31.810132 and 14137.323231. Quadratic regression and generalized additive models did not outperform these selections.

The spline models permit interpretation of smooth nonlinear St relationships, while random forests capture more flexible nonlinearities and interactions. We therefore investigate fitted response profiles and parameter contrasts separately for each selected model, using common comparisons across the four responses. These selections are based on predictive performance under the current five-fold validation design and remain subject to diagnostic and stability checks.

Shared bootstrap inference

Inference asks: how does St relate to each response; how do Re and Fr comparisons differ; and does a change in mean accompany a change in variability or tails? We inspect fitted functions and compare predictions at supported settings.

For each selected main model, ordinary bootstrap samples draw simulation rows with replacement. We refit the fixed specification and take 2.5th and 97.5th percentiles for exploratory 95% fit intervals and differences. These are pointwise intervals, conditional on selected settings. They omit model-selection uncertainty, are not corrected for multiple comparisons, and are not future prediction intervals. Because the settings are designed simulations, interpret the intervals as approximate stability evidence. Replicate-level information needed to quantify the original moment estimates' uncertainty is unavailable.

```
# Selected response-specific model families
selected_family <- c(mean = "spline", sd   = "spline", skew = "forest", kurt = "forest"
)

stopifnot(
  identical(names(selected_family), responses), all(selected_family %in% names(families))
)

main_ids <- unique(unname(selected_family))

groups <- unique(train[c("Re", "Fr", "Re_cat", "Fr_cat", "group")])
profile_grid <- do.call(rbind, lapply(seq_len(nrow(groups)), function(j) {
```

```
  group <- groups[j, ]; observed <- train$St[train$group == group$group]
  d <- group[rep(1, 60), ]; d$St <- seq(min(observed), max(observed), length.out = 60)
  d
}))
st_grid <- groups[rep(seq_len(nrow(groups)), each = 2), ]
st_grid$St <- rep(c(.2, 1), nrow(groups))
condition_grid <- groups; condition_grid$St <- .5
inference_grid <- rbind(profile_grid, st_grid, condition_grid)

set.seed(settings$bootstrap_seed)
bootstrap_rows <- if (isTRUE(params$run_inference))
  replicate(settings$bootstrap_replicates,
            sample.int(nrow(train), replace = TRUE), simplify = FALSE) else list()

profiles <- list(); st_comparisons <- list(); condition_comparisons <- list()
bootstrap_counts <- list()
if (isTRUE(params$run_inference)) for (id in main_ids) for (y in responses) {
  if (id != selected_family[[y]]) next

  selected <- fits[[id]][[y]]
  point <- families[[id]]$predict(selected$object, inference_grid)
  boot <- sapply(seq_along(bootstrap_rows), function(b) tryCatch({
    object <- families[[id]]$fit(train[bootstrap_rows[[b]], ], y,
                                 selected$configuration, 500L + b)
    p <- families[[id]]$predict(object, inference_grid)
    if (length(p) != nrow(inference_grid)) stop("Wrong bootstrap prediction length")
    p
  }, error = function(e) rep(NA_real_, nrow(inference_grid))))
  valid <- colSums(!is.finite(boot)) == 0
  key <- paste(id, y, sep = "_")
  bootstrap_counts[[key]] <- data.frame(family = id, response = y,
    attempted = settings$bootstrap_replicates, successful = sum(valid))
  if (sum(valid) < ceiling(.9 * settings$bootstrap_replicates))
    stop("Too many bootstrap fitting failures for ", key, "; review the specification.")
  boot <- boot[, valid, drop = FALSE]
  lower <- apply(boot, 1, quantile, .025); upper <- apply(boot, 1, quantile, .975)
  pi <- seq_len(nrow(profile_grid)); si <- nrow(profile_grid) + seq_len(nrow(st_grid))
  profiles[[key]] <- cbind(family = id, response = y, profile_grid,
                           estimate = point[pi], lower = lower[pi], upper = upper[pi])
  for (j in seq_len(nrow(groups))) {
    a <- si[2*j - 1]; b <- si[2*j]; delta <- boot[b, ] - boot[a, ]
    st_comparisons[[length(st_comparisons) + 1L]] <- cbind(family = id, response = y,
```

```
      groups[j, ], St_from = .2, St_to = 1, difference = point[b] - point[a],
      lower = quantile(delta, .025), upper = quantile(delta, .975), ratio = point[b]/point[a]
  }
  offset <- nrow(profile_grid) + nrow(st_grid)
  for (i in seq_len(nrow(groups) - 1)) for (j in seq.int(i + 1, nrow(groups))) {
    if (groups$Re[i] == groups$Re[j] || groups$Fr[i] == groups$Fr[j]) {
      pair <- c(i, j)
      pair <- if (groups$Re[i] == groups$Re[j]) pair[order(groups$Fr[pair], decreasing = TRUE
        pair[order(groups$Re[pair])]
      a <- offset + pair[1]; b <- offset + pair[2]; delta <- boot[b, ] - boot[a, ]
      condition_comparisons[[length(condition_comparisons) + 1L]] <- data.frame(
        family = id, response = y, St = .5, from_Re = groups$Re[pair[1]],
        to_Re = groups$Re[pair[2]], from_Fr = groups$Fr[pair[1]], to_Fr = groups$Fr[pair[2]]
        difference = point[b] - point[a], lower = quantile(delta, .025),
        upper = quantile(delta, .975), ratio = point[b]/point[a])
    }
  }
}
profile_table <- if (length(profiles)) do.call(rbind, profiles) else data.frame()
st_table <- if (length(st_comparisons)) do.call(rbind, st_comparisons) else data.frame()
condition_table <- if (length(condition_comparisons)) do.call(rbind, condition_comparisons) e
if (nrow(profile_table)) {
  write_result(profile_table, "response-profiles.csv")
  write_result(st_table, "st-comparisons.csv")
  write_result(condition_table, "condition-comparisons.csv")
  write_result(do.call(rbind, bootstrap_counts), "bootstrap-counts.csv")
}
```

Fitted relationships and response-specific interpretation

These panels show all observed regimes. Compare curves within panels and read the numerical axes before comparing panels: response axes are logarithmic and panel ranges differ. Shading is approximate pointwise fit uncertainty, not a prediction interval. Detailed plotted values are exported as CSV.

```
if (nrow(profile_table)) {
  colours <- setNames(grDevices::hcl.colors(length(main_ids), "Dark 3"), main_ids)
  if (all(c("spline", "forest") %in% main_ids)) {
    colours["spline"] <- "#146c94"; colours["forest"] <- "#bc5c24"
  }
  for (y in responses) {
```

```
    par(mfrow = c(2, 4), mar = c(4, 4, 3, 1))
    for (g in levels(train$group)) {
      observed <- train[train$group == g, ]
      selected <- profile_table[profile_table$response == y & profile_table$group == g, ]
      limits <- range(c(observed[[y]], selected$estimate, selected$lower, selected$upper))
      limits[1] <- max(limits[1], min(observed[[y]])/10)
      plot(observed$St, observed[[y]], log = "y", pch = 16, cex = .6,
           ylim = limits, xlab = "St", ylab = response_labels[y],
           main = paste("Re", observed$Re[1], "Fr", observed$Fr[1]))
      for (id in main_ids) {
        d <- selected[selected$family == id, ]
        polygon(c(d$St, rev(d$St)), c(pmax(d$lower, limits[1]), rev(pmax(d$upper, limits[1]))
                border = NA, col = adjustcolor(colours[id], alpha.f = .12))
        lines(d$St, d$estimate, col = colours[id], lwd = 2)
      }
      points(observed$St, observed[[y]], pch = 16, cex = .6)
      legend("topleft", main_ids, col = colours, lty = 1, bty = "n", cex = .65)
    }
  }
  par(mfrow = c(1, 1))
}
```

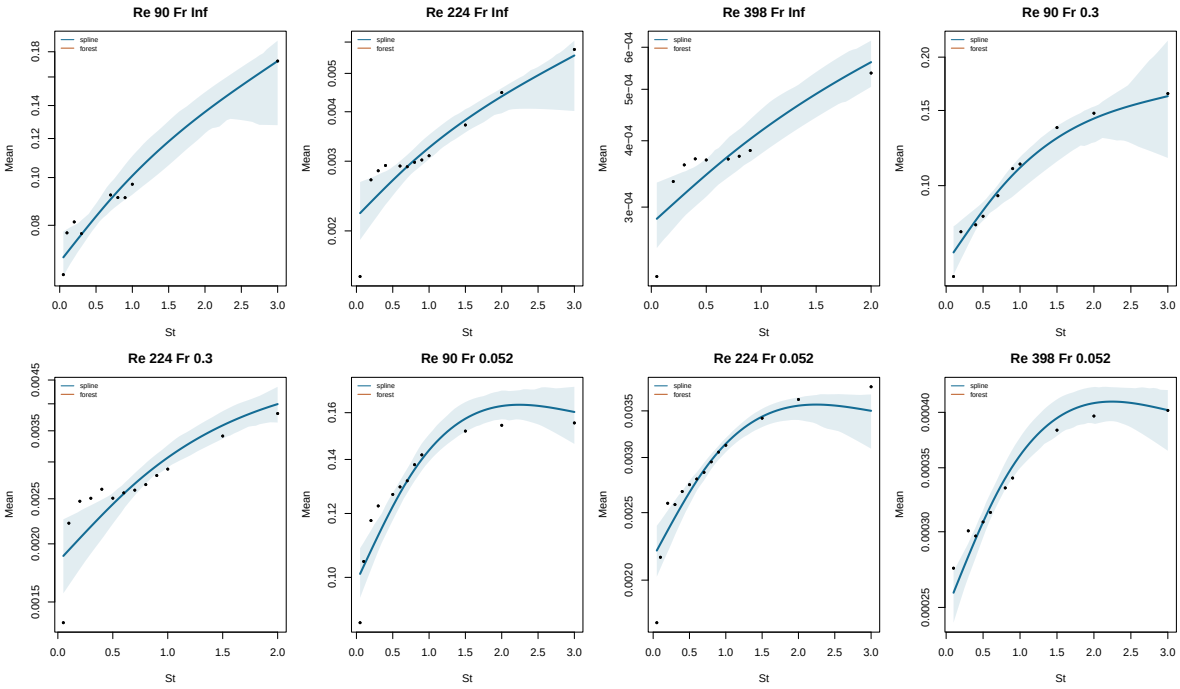


Figure 6: Response profiles by observed Re–Fr regime. Blue and orange identify the two current main families; additional families receive additional colours.

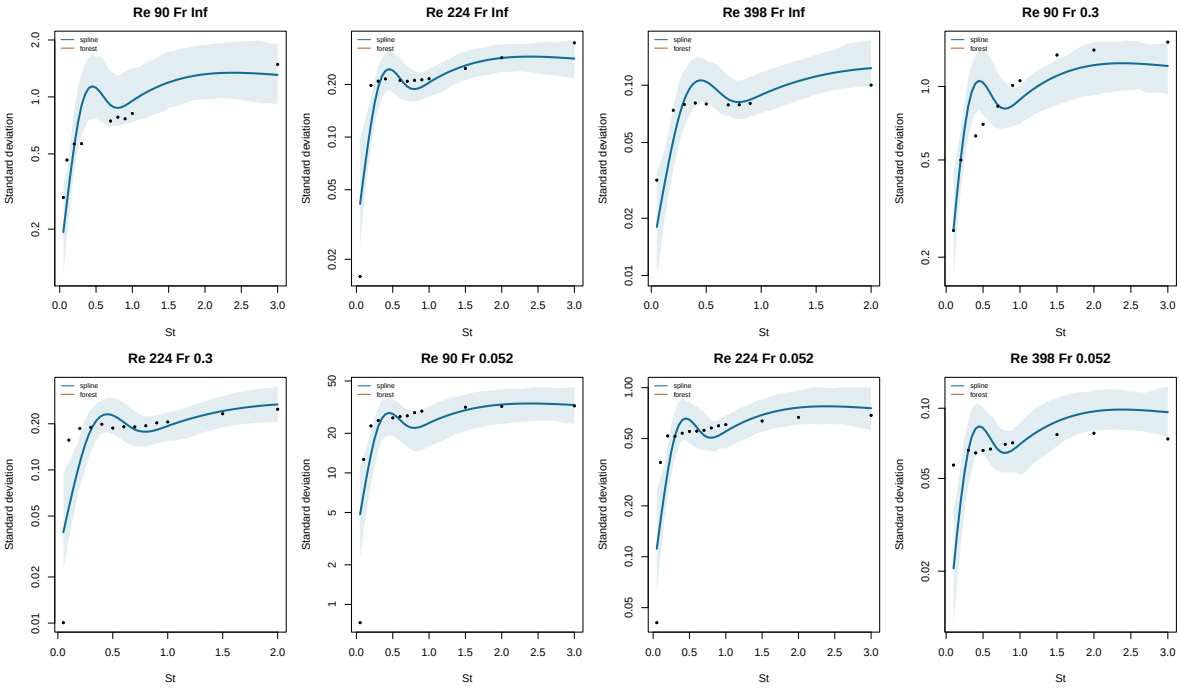


Figure 7: Response profiles by observed Re–Fr regime. Blue and orange identify the two current main families; additional families receive additional colours.

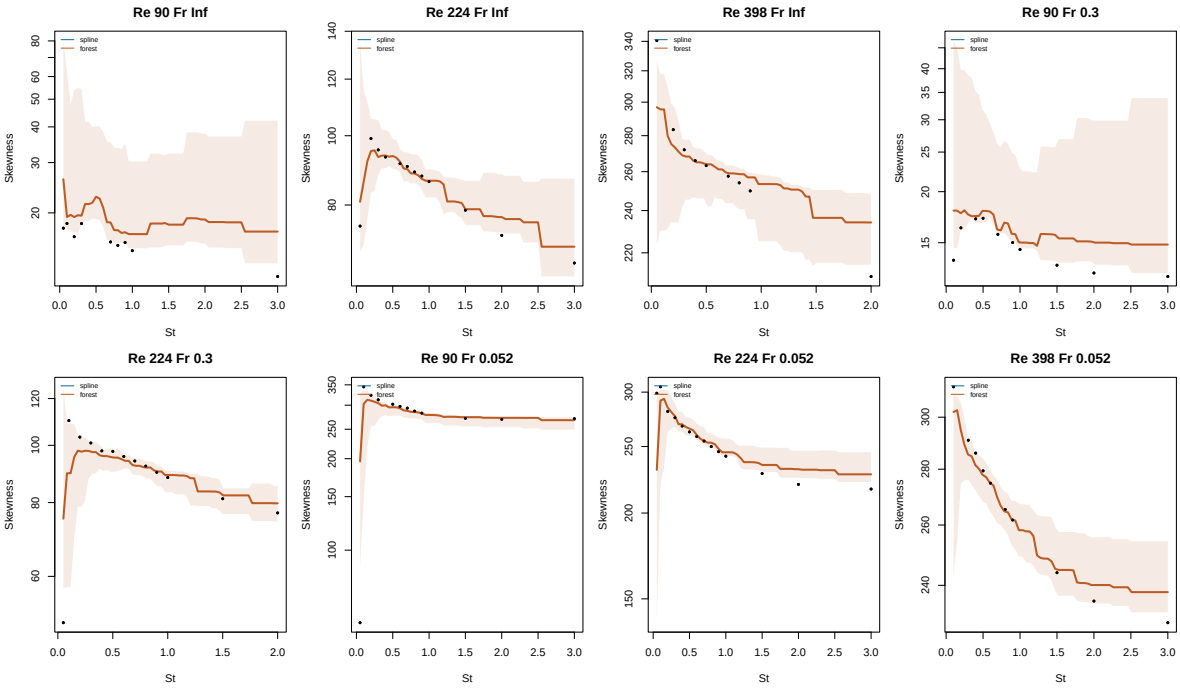


Figure 8: Response profiles by observed Re–Fr regime. Blue and orange identify the two current main families; additional families receive additional colours.

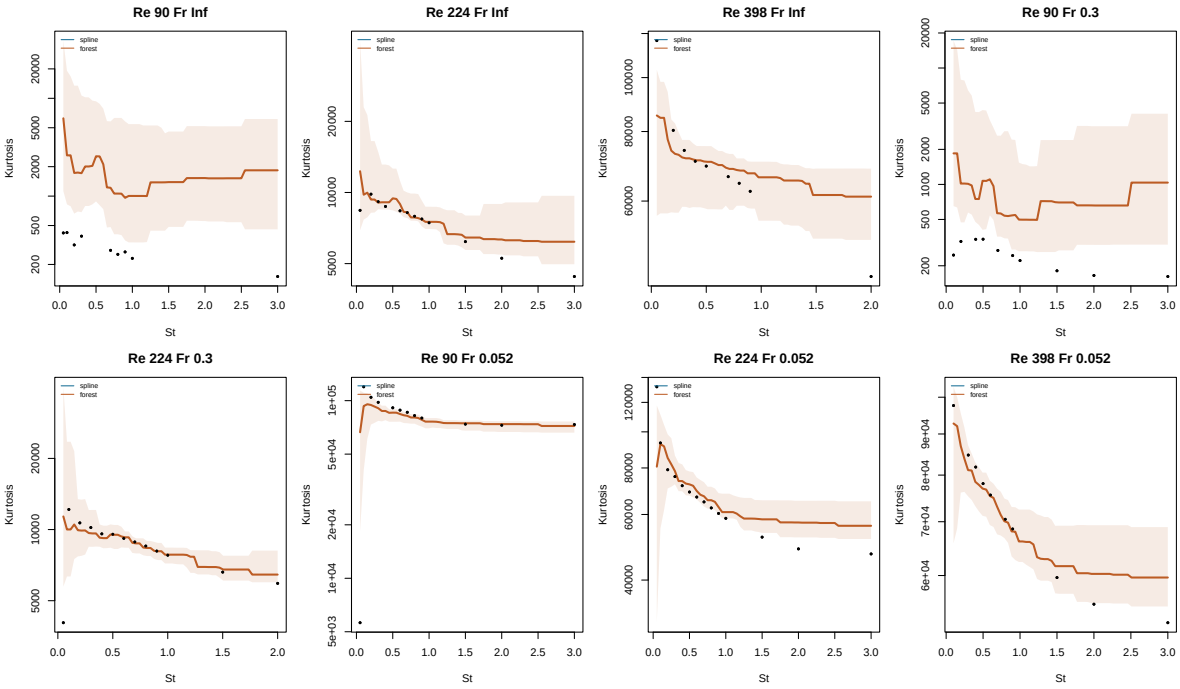


Figure 9: Response profiles by observed Re–Fr regime. Blue and orange identify the two current main families; additional families receive additional colours.

St: compare 0.2 with 1 within each observed regime

An interval for the difference entirely above zero supports a positive fitted comparison; one spanning zero is inconclusive under this exploratory procedure. Identical proportional changes across groups may be imposed by a shared log-scale curve. Do not present such repeated ratios as independent discoveries. A different selected formula across responses is not itself proof that their effects differ.

```
if (nrow(st_table)) show_table(st_table[c("family", "response", "Re", "Fr",
  "difference", "lower", "upper", "ratio")], digits = 6)
```

family	response	Re	Fr	difference	lower	upper	ratio
spline	mean	224	0.052	0.000782	0.000610	0.000941	1.331855
spline	mean	224	Inf	0.000881	0.000454	0.001147	1.372307
spline	mean	90	Inf	0.027357	0.013240	0.038660	1.372307
spline	mean	398	Inf	0.000113	0.000057	0.000161	1.372307
spline	mean	90	0.300	0.036258	0.020552	0.046997	1.490398
spline	mean	398	0.052	0.000090	0.000069	0.000111	1.331855

family	response	Re	Fr	difference	lower	upper	ratio
spline	mean	90	0.052	0.035819	0.027993	0.044528	1.331855
spline	mean	224	0.300	0.001009	0.000585	0.001290	1.490398
spline	sd	224	0.052	0.232980	0.138429	0.326016	1.732281
spline	sd	224	Inf	0.086656	0.046899	0.128068	1.732281
spline	sd	90	Inf	0.403038	0.221242	0.689198	1.732281
spline	sd	398	Inf	0.037610	0.020689	0.055632	1.732281
spline	sd	90	0.300	0.373718	0.204600	0.514382	1.732281
spline	sd	398	0.052	0.029642	0.016551	0.041341	1.732281
spline	sd	90	0.052	10.121561	5.330681	14.917849	1.732281
spline	sd	224	0.300	0.081820	0.047642	0.113690	1.732281
forest	skew	224	0.052	-39.604013	-55.955721	-15.613908	0.860948
forest	skew	224	Inf	-8.681453	-19.093211	3.928756	0.908799
forest	skew	90	Inf	-2.473700	-27.713767	5.101913	0.872042
forest	skew	398	Inf	-21.723272	-52.216717	3.539812	0.921055
forest	skew	90	0.300	-2.682089	-21.917579	3.151265	0.848591
forest	skew	398	0.052	-36.724496	-49.206560	-19.155723	0.875487
forest	skew	90	0.052	-31.947909	-49.215139	15.727398	0.897106
forest	skew	224	0.300	-8.865362	-17.820027	8.741016	0.909475
forest	kurt	224	0.052	-24213.933378	-37662.445868	-9470.341769	0.715474
forest	kurt	224	Inf	-1821.903229	-9058.324953	-466.155449	0.804801
forest	kurt	90	Inf	-720.775878	-11705.451838	401.615761	0.582086
forest	kurt	398	Inf	-7710.119391	-17800.548356	316.324181	0.895681
forest	kurt	90	0.300	-518.729803	-7466.225673	557.922058	0.489997
forest	kurt	398	0.052	-20765.309014	-29682.440192	-10523.840301	0.761094
forest	kurt	90	0.052	-18352.181986	-28313.011337	-539.300895	0.805901
forest	kurt	224	0.300	-2106.842662	-5644.055827	-12.268703	0.787964

Re and Fr: supported comparisons at St = 0.5

Each comparison changes Re at fixed Fr, or changes Fr at fixed Re. No comparison uses the absent regime. These results answer whether apparent gravity effects depend on the flow regime and whether mean, spread and tails respond similarly.

```
if (nrow(condition_table)) {
  # Display gravity comparisons in the notebook; the CSV also includes Re comparisons.
```

```
  gravity_comparisons <- condition_table[condition_table$from_Re == condition_table$to_Re, ]
  show_table(gravity_comparisons, digits = 6)
}
```

family	response	St	from_Re	to_Re	from_Fr	to_Fr	difference	lower	upper	ratio
spline	mean	0.5	224	224	Inf	0.052	-0.000011	-0.000269	0.000195	0.995764
spline	mean	0.5	224	224	0.3	0.052	0.000240	0.000043	0.000447	1.098606
spline	mean	0.5	224	224	Inf	0.300	-0.000251	-0.000555	0.000017	0.906388
spline	mean	0.5	90	90	Inf	0.300	0.004103	-0.003824	0.010815	1.049206
spline	mean	0.5	90	90	Inf	0.052	0.039042	0.033214	0.045962	1.468202
spline	mean	0.5	398	398	Inf	0.052	-0.000038	-0.000064	-0.000005	0.889490
spline	mean	0.5	90	90	0.3	0.052	0.034939	0.029296	0.041253	1.399346
spline	sd	0.5	224	224	Inf	0.052	0.405968	0.261542	0.568949	2.688559
spline	sd	0.5	224	224	0.3	0.052	0.419386	0.256563	0.585433	2.847475
spline	sd	0.5	224	224	Inf	0.300	-0.013418	-0.082772	0.061683	0.944190
spline	sd	0.5	90	90	Inf	0.300	-0.081347	-0.595408	0.236378	0.927252
spline	sd	0.5	90	90	Inf	0.052	26.963536	18.061436	35.195892	25.113177
spline	sd	0.5	398	398	Inf	0.052	-0.022106	-0.057540	0.002415	0.788143
spline	sd	0.5	90	90	0.3	0.052	27.044883	18.252105	35.395093	27.083446
forest	skew	0.5	224	224	Inf	0.052	171.798994	163.105961	183.500572	2.834029
forest	skew	0.5	224	224	0.3	0.052	169.926377	163.397683	180.476493	2.778484
forest	skew	0.5	224	224	Inf	0.300	1.872617	-5.648721	5.138598	1.019991
forest	skew	0.5	90	90	Inf	0.300	-4.823643	-18.684882	1.584750	0.788269
forest	skew	0.5	90	90	Inf	0.052	272.600545	235.440351	277.387287	12.965651
forest	skew	0.5	398	398	Inf	0.052	13.545706	3.126360	31.742907	1.051236
forest	skew	0.5	90	90	0.3	0.052	277.424189	256.345950	281.683413	16.448261
forest	kurt	0.5	224	224	Inf	0.052	62835.614135	57471.805206	69096.519437	1.651941
forest	kurt	0.5	224	224	0.3	0.052	62754.022886	57817.169635	69088.261167	8.586413
forest	kurt	0.5	224	224	Inf	0.300	81.591250	-2549.239325	744.913061	1.008637
forest	kurt	0.5	90	90	Inf	0.300	-1479.670763	-5235.142805	278.212741	0.420681

family	response	St	from_Re	to_Re	from_Fr	to_Fr	difference	lower	upper	ratio
forest	kurt	0.5	90	90	Inf	0.052	83061.625657	70233.361718	96667.613533	3.520207
forest	kurt	0.5	398	398	Inf	0.052	6295.363366	-737.913071	18221.797344	1.089074
forest	kurt	0.5	90	90	0.3	0.052	84541.296427	75636.202628	98476.489729	9.680866

Current findings to revisit when adding models

For the current spline specification, increasing St from 0.2 to 1 increases fitted mean volume across the observed regimes. Both current families associate strong gravity with larger mean volume at Re = 90 and St = 0.5. The spline instead suggests a decrease at Re = 398; the forest comparison is uncertain there. Tail comparisons are more sensitive to the family, and the selected shared spline curves do not establish a clear St change in skewness or kurtosis over that interval.

These are fitted relationships, not established causal laws. St represents particle inertia and Fr relative gravity influence, so condition-dependent relationships are physically plausible. The supplied slides note changing simulation domains/resolution. Confirm volume normalisation before interpreting large between-Re differences as purely physical effects. Treat the paragraphs here as interpretation to update after reviewing new families, not as results that additional models must reproduce.

Forest importance

The lecture's regression-forest importance is the decrease in RSS attributed to splits on a predictor, averaged over trees. It indicates predictive use within an outcome; it gives neither direction nor causal importance. Scores from different response scales should not be compared, and available split choices can affect importance. Supported fitted comparisons above supply direction.

```
if ("forest" %in% names(fits)) {
  importance <- do.call(rbind, lapply(responses, function(y) {
    x <- fits$forest[[y]]$object$model$variable.importance
    data.frame(response = y, predictor = names(x), RSS_importance = as.numeric(x))
  }))
  write_result(importance, "forest-importance.csv")
  show_table(importance, digits = 5)
}
```

response	predictor	RSS_importance
mean	St	2.481000e-02
mean	Re_cat	2.348100e-01
mean	Fr_cat	9.120000e-03
sd	St	7.637076e+02
sd	Re_cat	2.843958e+03
sd	Fr_cat	3.045244e+03
skew	St	6.944919e+04
skew	Re_cat	3.537009e+05
skew	Fr_cat	6.658597e+05
kurt	St	1.170900e+10
kurt	Re_cat	3.183865e+10
kurt	Fr_cat	7.437157e+10

Prediction exports and team decision

Each family produces one CSV with the original test inputs and four predictions, in original row order. Separate response fits do not guarantee that all four outputs jointly describe a possible distribution. We report that limitation; no automatic moment adjustment is applied. Confirm whether the final submission requires SD or variance and the exact headers before using a candidate CSV.

```
for (id in names(families)) {
  out <- test_raw[c("St", "Re", "Fr")]
  for (y in responses) out[[y]] <- fits[[id]][[y]]$test_prediction
  stopifnot(nrow(out) == nrow(test_raw),
    identical(out[c("St", "Re", "Fr")], test_raw[c("St", "Re", "Fr")]))
  write_result(out, paste0("predictions-", id, ".csv"))
}

# Edit only after the team's review of all registered families.
# A family can legitimately be selected for more than one response.
final_choice <- c(mean = NA_character_, sd = NA_character_,
                  skew = NA_character_, kurt = NA_character_)
if (all(!is.na(final_choice))) {
  stopifnot(identical(names(final_choice), responses),
            all(final_choice %in% names(families)))
  final <- test_raw[c("St", "Re", "Fr")]
  for (y in responses) final[[y]] <- fits[[final_choice[[y]]]][[y]]$test_prediction
  write_result(final, "predictions-team-selected.csv")
```

```
  write_result(data.frame(response = responses, family = final_choice), "team-selection.csv")
}
```

Extension and review checklist

- Register a family before `evaluate-families`; ensure its prediction function returns original-scale values in input row order. The quadratic example shows the interface. Keep its grid modest for this small dataset.
- Rerender to bring it into the common CV table, residual plots, bootstrap, supported contrasts and prediction exports. Do not import scores from different folds as if they were directly comparable.
- Review selected specifications and candidate failures. Do not infer significance from the smallest CV error, a selected interaction, or an importance ranking.
- Update the scientific interpretation after comparing new families. Check the same questions for each response rather than writing unrelated model summaries.
- Agree final families per response; fill in `final_choice` only after that review. Model-count diversity is not a reason to force a worse family onto a response.
- For the later report, select the most informative diagnostics, comparisons and figures. This working file retains detail and is not subject to the report's four-page presentation limit.

The remaining limits are the small designed dataset, missing regime, boundary prediction risk, residual misfit, approximate fit uncertainty, response-scale choices, and possible inconsistency between separately predicted summaries. None of the candidate files has been submitted automatically.

```
saveRDS(list(settings = settings, folds = fold,
             configurations = lapply(fits, function(family)
               lapply(family, function(x) x$configuration))),
        file.path(results_dir, "analysis-settings.rds"))
capture.output(sessionInfo(), file = file.path(results_dir, "session-info.txt"))
```